



Patrones arquitectónicos: fundamentos

Diseño y Patrones Arquitectónicos · Jesús Ariel González Bonilla · 2026-B · Corte 1 · Unidad 1

Fundamentos de patrones arquitectónicos y su clasificación

El sistema funciona y nadie lo toca

¿Cuántos sistemas conocen que cumplen todo lo que piden... y que nadie se atreve a modificar?

- **Cumple los requisitos funcionales**
- **Un cambio pequeño toma semanas**
- **Nadie quiere desplegar un viernes**

Lo que vas a poder hacer al terminar

- **Explicar por qué la arquitectura decide**
- **Escribir atributos de calidad con medida**
- **Separar patrón arquitectónico de diseño**
- **Clasificar patrones clásicos y modernos**
- **Diagnosticar el reto de tu equipo**

01

1. Por qué la arquitectura decide

La arquitectura es plural

Arquitectura: las estructuras necesarias para razonar sobre el sistema, sus elementos, relaciones y propiedades.

Módulos

cómo se divide el código

Componentes

qué corre y cómo se comunica

Despliegue

dónde corre cada parte

Arquitectura es lo caro de revertir

Decisión	¿Arquitectónica?	Por qué
Tres capas con dependencias en un sentido	Sí	Cambiarla reorganiza todo el código
Eventos asíncronos en vez de llamadas	Sí	Cambia errores, consistencia y despliegue
Un for o un stream en una lista	No	Se cambia en minutos
Nombre de una clase interna	No	Un refactor local lo resuelve

Todo en arquitectura es un compromiso

Primera ley de Richards y Ford (2020): en arquitectura de software, todo es un compromiso.

- **Ningún patrón es «el mejor»**
- **Cada uno favorece unos atributos**
- **Y castiga otros a cambio**
- **Tu trabajo: qué sacrificar y por qué**

¿Qué hace arquitectónica una decisión?

- A** Que aparezca en un diagrama UML del proyecto
- B** Que revertirla después sea costoso o afecte a todo el sistema
- C** Que la tome el arquitecto y no el equipo
- D** Que involucre elegir un lenguaje de programación

02

2. Conceptos fundamentales

Componentes, conectores y reglas

Componente

unidad con una
responsabilidad

Conector

HTTP, cola de
mensajes, tubería

Configuración

cómo se conectan
entre sí

Restricción

regla que la
arquitectura impone

Los atributos de calidad eligen

Atributo	Pregunta que responde
Rendimiento	¿Responde a tiempo bajo la carga esperada?
Escalabilidad	¿Soporta más usuarios agregando recursos?
Disponibilidad	¿Está operativo cuando se necesita?
Modificabilidad	¿Cuánto cuesta cambiarlo?
Seguridad	¿Resiste usos no autorizados?
Desplegabilidad	¿Sale a producción seguido y con poco riesgo?

«Que sea rápido» no decide nada

Parte del escenario	Ejemplo: Mercado Campesino
Fuente	300 clientes
Estímulo	Piden al mismo tiempo el jueves antes del cierre
Artefacto	Servicio de pedidos
Entorno	Operación normal, día de cierre de pedidos
Respuesta	Cada pedido se registra y confirma
Medida	El 95 % se confirma en menos de 2 s

¿Cuál es un escenario de calidad útil?

- A** El sistema debe ser rápido y escalable
- B** El sistema debe usar microservicios
- C** Con 300 usuarios a la vez, el 95 % de los pedidos se confirma en menos de 2 s
- D** El sistema debe tener la mejor arquitectura posible

Cinco principios que no envejecen

Principio	Qué evita
Separación de responsabilidades	Que la interfaz arrastre reglas de negocio
Modularidad	Que todo dependa de todo
Bajo acoplamiento	Cambios en cascada
Alta cohesión	Una función repartida en diez lugares
Abstracción y ocultamiento	Detalles técnicos filtrados a todo el código

03

3. Patrón, estilo y patrón de diseño

Contexto, problema y solución

Un patrón arquitectónico es una solución probada a un problema recurrente de organización del sistema completo.

Contexto

dónde aparece el problema



Problema

qué fuerzas hay que equilibrar



Solución

organización probada y reutilizable

Tres niveles, tres alcances de error

Patrón arquitectónico · organiza el sistema completo

Patrón de diseño · organiza clases dentro de un componente

Idioma · organiza unas líneas en un lenguaje

Estilo y patrón: casi sinónimos

Estilo

- Familia de organización
- Elementos y restricciones
- Shaw y Garlan (1996)

Patrón

- Solución en un contexto
- Contexto, problema, solución
- Buschmann et al. (1996)

MVC vive en dos niveles

Como patrón arquitectónico, MVC organiza la presentación de toda la aplicación. Como patrón de diseño, tres objetos que colaboran.

- **Su origen está en Smalltalk**
- **En un informe, aclara el nivel**

Strategy, Observer y Factory son:

- A** Patrones arquitectónicos clásicos
- B** Patrones arquitectónicos modernos
- C** Patrones de diseño que organizan clases dentro de un componente
- D** Estilos arquitectónicos

04

4. Clasificación: clásicos y modernos

Cinco patrones clásicos

Patrón	Idea central	Favorece	Cuesta
Capas	Niveles apilados; cada uno usa el de abajo	Mantenibilidad	Latencia entre capas
Cliente-servidor	El cliente pide, el servidor provee	Centralización, seguridad	Punto único de fallo
MVC	Modelo, vista y controlador separados	Varias vistas, pruebas	Piezas de más en lo simple
Pipes and filters	Datos que fluyen por filtros	Reordenar y reutilizar pasos	Poco interactivo
Peer-to-peer	Nodos iguales: cliente y servidor	Escalar sin centro	Seguridad y consistencia

Los clásicos comparten cuatro rasgos

Desacoplamiento

partes independientes con interacciones definidas

Organización modular

capas o módulos con una responsabilidad

Protocolos claros

HTTP, tuberías unidireccionales

Reusabilidad

se aplican en muchos dominios

Modernos: cómo se reparte el sistema

Patrón	Idea central	Favorece	Cuesta
Microservicios	Servicios autónomos con datos propios	Desplegar y escalar por partes	Operación y consistencia
Serverless	Funciones que el proveedor ejecuta	Pagar solo por uso	Arranque en frío, dependencia
Cloud-native	Contenedores, orquestación, autoescalado	Elasticidad, autorreparación	Curva y costo de plataforma

Modernos: cómo se organiza la lógica

Patrón	Idea central	Favorece	Cuesta
Hexagonal	El dominio al centro, lo externo por puertos	Probar, cambiar tecnología	Interfaces y mapeos
Event-driven	Componentes que reaccionan a eventos	Desacoplar en el tiempo	Flujo difícil de seguir
CQRS	Modelos separados para escribir y leer	Lecturas rápidas	Dos modelos que sincronizar

Los modernos llevan todo al extremo

Modularidad extrema

actualizar una parte sin tocar
el resto

Elasticidad

escalar solo con la carga

Asincronía

eventos en vez de esperas

Clásicos frente a modernos

Clásicos

- Se despliega junto
- Comunicación síncrona
- Infraestructura fija
- Un fallo tumba el sistema

Modernos

- Se despliega por partes
- Comunicación asíncrona
- Infraestructura elástica
- Un fallo degrada una parte

Lo moderno no reemplaza lo clásico

**Un microservicio por dentro suele ser capas o hexagonal.
Por eso en el proyecto primero haces una v1 clásica.**

Microservicio · la frontera de despliegue

Capas o hexagonal · la organización interna

Base de datos propia · los datos del servicio

Leer, limpiar, validar y cargar es...

A Cliente-servidor

B Pipes and filters

C Peer-to-peer

D MVC

¿Qué distingue a los modernos?

- A** Que no usan principios como el bajo acoplamiento
- B** Que se apoyan en infraestructura elástica y asincronía para escalar y resistir fallos
- C** Que siempre son más eficientes que los clásicos
- D** Que reemplazan por completo a los clásicos

05

5. Beneficios y desafíos

Cada beneficio trae su desafío

Beneficio que se busca	Desafío que aparece
Separar para cambiar más rápido	Más piezas e interfaces
Escalar solo una parte	Latencia, fallos parciales, seguridad
Desplegar sin coordinar	Versionar contratos entre servicios
Desacoplar con eventos	Consistencia eventual, depuración difícil
Pagar solo por uso	Dependencia del proveedor

Dos compromisos que siempre vuelven

Compromiso	Si priorizas lo primero	Si priorizas lo segundo
Acoplamiento o flexibilidad	Menos piezas, cambios en cascada	Más piezas, cambios aislados
Consistencia o disponibilidad	Datos siempre al día, más esperas	Respuesta siempre, datos que tardan

Moderno no es sinónimo de mejor

Un patrón moderno en un sistema pequeño, con un equipo pequeño, suele costar más de lo que aporta.

- **La red no es confiable**
- **La latencia no es cero**
- **El ancho de banda se acaba**
- **Pregunta: ¿qué atributo justifica el costo?**

Un costo típico de los microservicios:

- A** No permiten escalar componentes por separado
- B** Más complejidad operativa y datos difíciles de mantener consistentes
- C** Obligan a desplegar todo el sistema junto
- D** Impiden usar bases de datos

06

6. Cómo leer un sistema real

Cinco preguntas para clasificar

1

Piezas

componentes y
conectores

2

Lógica

dónde vive el
negocio

3

Despliegue

una unidad o
varias

4

Atributo

el que más
castiga un
error

5

Patrones

el dominante y
los que
acompañan

Mercado Campesino: el caso

Una asociación de productores de Neiva recibe pedidos por la web y entrega dos veces por semana.

Navegador

Aplicación web

páginas y lógica
mezcladas

Base de datos

Hoja de cálculo

rutas a mano

El problema no son los microservicios

Pregunta	Respuesta
Piezas	Navegador, aplicación web, base de datos
Lógica	En el servidor, mezclada con las páginas
Despliegue	Una sola unidad
Atributo que manda	Modificabilidad y disponibilidad el día de pedidos
Patrones	Cliente-servidor; MVC incompleto; sin capas

Una v1 en capas resuelve lo urgente

Presentación (MVC) · páginas y controladores

Negocio · precios, pedidos y rutas

Datos · repositorios y base de datos

¿Qué le falta a Mercado Campesino?

- A** Microservicios
- B** Despliegue en la nube
- C** Capas: las reglas viven mezcladas con las páginas
- D** Una base de datos no relacional

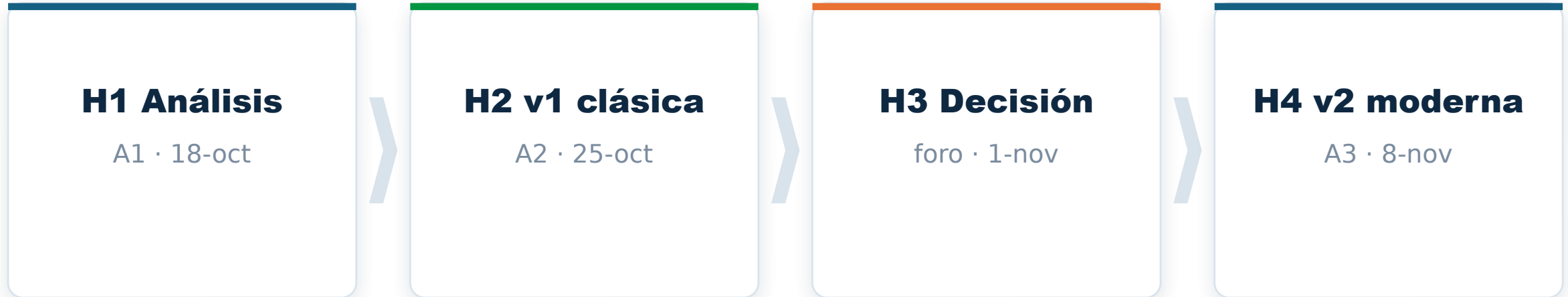
Patrones en sistemas que ya usas

Sistema real	Patrón dominante
Navegador y servidor web	Cliente-servidor
Compilador; tuberías de Unix	Pipes and filters
BitTorrent	Peer-to-peer
Spring MVC, Django, Rails	MVC sobre capas
ERP en tres capas	Capas
Funciones en la nube (AWS Lambda)	Serverless
Streaming de eventos (Apache Kafka)	Event-driven

07

7. El proyecto de aula del curso

Cuatro actividades, un solo proyecto



- **Mismo reto en todo el curso**
- **Ninguna actividad del aula cambia**
- **La Guía del proyecto de aula lo explica**

La v2 se compara con tu v1

La pregunta difícil de la A3 —compara con otras alternativas— solo tiene buena respuesta si la alternativa existe.

- **Tu v1 existe y se ejecuta**
- **Etiquétala en git antes de la v2**
- **Mide al menos tres criterios**

Cinco retos del Huila

Reto	Problema	v1 sugerida	v2 candidatas
TrazaCafé	Trazabilidad de café especial	Capas	Event-driven, CQRS
RutaSur	Reservas de turismo regional	MVC sobre capas	Microservicios, eventos
AquaBetania	Sensores en estanques piscícolas	Pipes and filters	Event-driven, serverless
CitaSalud	Citas en una red de IPS	Capas con MVC	Hexagonal, CQRS
PQRS Ciudadana	Peticiones de una alcaldía	MVC sobre capas	Hexagonal, eventos

El FO-IV-159 cierra el proyecto

- **Un informe por equipo**
- **Cuenta en el 40 % de la A3**
- **Anexo 2: documento técnico y repositorio**
- **Identificaciones solo en el aula**
- **Formato y plantilla en la Guía**

Para el martes 13: equipo y reto

1

Formar equipo

2 o 3 integrantes

2

Elegir reto

uno del catálogo

3

Delimitar alcance

si el reto se repite

4

Avisar al tutor

mensaje por el aula

08

8. Cierre

Del concepto al Hito 1

La A1 pide

Con tu reto

Introducción

Por qué la arquitectura decide en este reto

Definición y clasificación

Clásicos vs modernos frente a sus atributos

Análisis de tres clásicos

Tres candidatos con sistema real de referencia

Reflexión final

Qué patrón para la v1 y qué riesgos

Cuatro errores que cuestan nota

- **Definir sin aplicar al reto**
- **Atributos de calidad sin medida**
- **Un solo patrón para todo**
- **Blogs contados como fuente académica**

¿Qué es la A1 dentro del proyecto?

- A** La primera versión del sistema funcionando
- B** Un diagnóstico arquitectónico individual del reto del equipo
- C** Un debate en el foro sobre patrones modernos
- D** El informe FO-IV-159 completo

Tres ideas para llevarse

Lo caro de revertir

eso es arquitectura

Atributos con medida

ellos eligen el patrón

Clásico y moderno

se combinan, no se
reemplazan

Bibliografía de la unidad

- **Bass, Clements y Kazman (2021)**
- **Richards y Ford (2020)**
- **Buschmann et al. (1996)**
- **Fowler (2002)**
- **Gamma et al. (1994)**



¿Preguntas?

Jesús Ariel González Bonilla · ariel5253@hotmail.com